

Projektdokumentation

FitFridge



Adaptive Systems and Artificial Intelligence
Media Systems SoSe 2026
Department Medientechnik der Fakultät DMI
Prof. Dr. Larissa Putzar, Sune Maute, Jörg Balzer

Gruppe 06
Sriraam Sinnarajah 2646750
Vu Thanh Tung Paeper 2761945

Inhaltsverzeichnis

1	Einleitung	3
1.1	Ausgangslage und Zielsetzung	3
1.2	Forschungsfrage	4
1.3	Aufbau des Berichts	4
2	Grundlagen und Einordnung	4
2.1	Vergleichbare Anwendungen	4
2.2	Large Language Models und Prompt Engineering	5
2.3	Kontextgestützte Generierung	6
2.4	Adaptive Systeme und MAPE-K	7
3	Projektkontext und technische Grundlage	8
3.1	Anforderungen an den ASaAI-Teil	8
3.2	Vorgehensmodell und Aufgabenverteilung	9
3.3	Tech-Stack und Datenquellen	9
4	Entwurf und Implementierung	10
4.1	ASaAI-Systemarchitektur	10
4.2	KI-basierte Nährwertschätzung	12
4.3	KI-gestützter Freestyle-Rezeptgenerator	13
4.4	Prompt-Aufbau und Kontextinjektion	14
4.5	Retry-Schleife, Makro-Reparatur und Validierung	15
5	Evaluation und Qualitätssicherung	17
5.1	Teststrategie	17
5.2	Bewertung der ASaAI-Funktionen	18
5.3	Grenzen der Evaluation	18
6	Diskussion	19
6.1	Bewertung des hybriden Ansatzes	19
6.2	Risiken und Verbesserungspotenziale	19
7	Fazit	20
7.1	Beantwortung der Forschungsfrage	20
7.2	Lessons Learned	20
	Literaturverzeichnis	22
	Abbildungsverzeichnis	22

1 Einleitung

1.1 Ausgangslage und Zielsetzung

Ernährungs- und Fitness-Apps wie Yazio oder MyFitnessPal unterstützen Nutzerinnen und Nutzer beim Erfassen von Lebensmitteln, Kalorien und Makronährstoffen. Yazio bietet unter anderem Kalorienzählen, Nährstofftracking und Barcode-Scanning an.¹ MyFitnessPal beschreibt sich ebenfalls als Anwendung zum Tracken von Kalorien, Makros, Ernährung und Fitnesszielen.² Diese Anwendungen sind vor allem dann hilfreich, wenn Mahlzeiten bereits geplant oder gegessen wurden.

Im Alltag entsteht jedoch häufig eine andere Situation: Es sind noch bestimmte Kalorien und Makronährstoffe offen, gleichzeitig sind nur bestimmte Lebensmittel im Haushalt vorhanden. Daraus ergibt sich die Frage, welches Gericht mit den vorhandenen Zutaten noch sinnvoll zubereitet werden kann. Genau an dieser Stelle setzt FitFridge an.

FitFridge wurde als Webanwendung entwickelt, die einen digitalen Kühlschrank, Mahlzeiten-Tracking und KI-gestützte Funktionen verbindet. Im digitalen Kühlschrank werden Lebensmittel mit Mengen und Nährwerten gespeichert. Der ASaAI-Teil der Anwendung umfasst zwei Funktionen: eine KI-Schätzung, die bei der Produktsuche neben den OpenFoodFacts-Treffern eine geschätzte Nährwertangabe für den eingegebenen Suchbegriff liefert, und einen KI-gestützten Freestyle-Rezeptgenerator, der aus dem aktuellen Kühlschrankbestand und den vorgegebenen Makrozielen passende und kulinarisch logische Rezepte erzeugt.

Das Projekt wurde modulübergreifend im Kontext der Module Software Engineering und Adaptive Systems and Artificial Intelligence umgesetzt. Der Software-Engineering-Teil umfasst unter anderem eine einfache Authentifizierung für strikt getrennte Nutzerdaten, die Kühlschrankverwaltung, eine Produktsuche über OpenFoodFacts und einen Mahlzeiten-Tracker. Dieser Bericht konzentriert sich auf den ASaAI-Teil. Funktionen aus dem Software-Engineering-Bereich werden nur dort erläutert, wo sie für das Verständnis der KI-Funktionen notwendig sind.

Ziel des ASaAI-Teils ist es, ein lokal ausgeführtes Large Language Model so in FitFridge einzubinden, dass daraus ein praktischer Mehrwert für die Rezeptplanung entsteht. Die Anwendung soll nicht nur Lebensmittel dokumentieren, sondern adaptiv vorhandene Lebensmittel, Makroziele und Rezeptgenerierung miteinander verbinden.

¹ Vgl. Yazio GmbH: Calorie Counter App, online verfügbar unter: <https://www.yazio.com/en/calorie-counter>, abgerufen am 12.07.2026.

² Vgl. MyFitnessPal: Calorie Tracker & Macro Tracking, online verfügbar unter: <https://www.myfitnesspal.com/>, abgerufen am 12.07.2026.

1.2 Forschungsfrage

Für den ASaAI-Teil ergibt sich folgende Forschungsfrage:

Wie können ein lokal ausgeführtes Large Language Model und regelbasierte Backend-Validierung kombiniert werden, um in einer Ernährungs-Webanwendung plausible Rezeptvorschläge und Nährwertschätzungen auf Basis vorhandener Lebensmittel zu erzeugen?

Die Frage umfasst beide KI-Funktionen von FitFridge. Einerseits wird betrachtet, wie ein Sprachmodell für Nährwertschätzungen eingesetzt werden kann. Andererseits geht es um die Generierung von Rezeptvorschlägen aus Kühlschrankdaten und Makrozielen. Wichtig ist dabei nicht nur die Generierung selbst, sondern auch die Kontrolle der Modellantworten durch das Backend.

1.3 Aufbau des Berichts

Kapitel 2 ordnet FitFridge in den Kontext vergleichbarer Ernährungs- und Tracking-Anwendungen ein und erläutert die relevanten Grundlagen zu Large Language Models, Prompt Engineering und kontextgestützter, RAG-ähnlicher Generierung sowie MAPE-K. RAG ist relevant, weil die Kühlschrankdaten als externer Kontext in den Prompt injiziert werden; MAPE-K dient als Referenzmodell für die adaptive Makro-Reparatur im Backend. Kapitel 3 beschreibt Anforderungen, Arbeitsweise, Aufgabenverteilung, Technologien und Datenquellen. Kapitel 4 stellt die Umsetzung der ASaAI-Funktionen dar. Kapitel 5 beschreibt Tests und Evaluation. Kapitel 6 diskutiert Risiken und Verbesserungspotenziale. Kapitel 7 beantwortet abschließend die Forschungsfrage.

2 Grundlagen und Einordnung

2.1 Vergleichbare Anwendungen

Yazio und MyFitnessPal dienen als Vergleichsanwendungen, weil beide Apps ähnliche Grundfunktionen im Bereich Ernährungstracking anbieten. Dazu gehören Lebensmittelsuche, Kalorienzählen, Makrotracking und Barcode-Scanning.³⁴ FitFridge übernimmt diese Grundidee des Trackings, erweitert sie jedoch um eine andere Perspektive: Die App berücksichtigt nicht nur, was gegessen wurde, sondern auch, welche Lebensmittel aktuell verfügbar sind.

Der zentrale Unterschied liegt damit in der Nutzung der gespeicherten Lebensmitteldaten. In klassischen Tracking-Apps werden Lebensmittel hauptsächlich

³ Vgl. Yazio GmbH: Calorie Counter App, online verfügbar unter: <https://www.yazio.com/en/calorie-counter>, abgerufen am 12.07.2026.

⁴ Vgl. MyFitnessPal: Calorie Tracker & Macro Tracking, online verfügbar unter: <https://www.myfitnesspal.com/>, abgerufen am 12.07.2026.

dokumentiert. In FitFridge bilden die gespeicherten Kühlschrankprodukte zusätzlich die Grundlage für KI-gestützte Rezeptvorschläge. Dadurch entsteht ein stärker planender Ansatz.

FitFridge kombiniert einen digitalen Kühlschrank, einen KI-gestützten Rezeptplaner und einen Mahlzeiten-Tracker. Nur wenn die Produktsuche keinen passenden Eintrag in der OpenFoodFacts-Datenbank findet, kann auf die KI-Schätzung in der Suche zurückgegriffen werden, die besonders bei unverarbeiteten Lebensmitteln wie Obst, Gemüse oder Fleisch genaue Näherungen liefert. Der Mahlzeiten-Tracker ist zudem direkt mit dem Kühlschrank verzahnt: Produkte können unmittelbar aus dem Kühlschrankbestand zu einer Mahlzeit hinzugefügt werden, wobei der Bestand automatisch angepasst wird. Der aktuelle Kühlschrankinhalt sowie die gesetzten Makroziele bilden dabei die Grundlage für die Rezeptgenerierung. Auf dieser Basis werden Rezeptvorschläge erzeugt, bei denen Zutaten, Mengen und Nährwerte an die verfügbaren Lebensmittel und die individuellen Zielwerte angepasst werden.

FitFridge richtet sich vor allem an fitnessinteressierte Nutzerinnen und Nutzer, die Kalorien und Makronährstoffe bewusst steuern möchten. Dazu gehören beispielsweise Muskelaufbau, Diät, Steuerung des Körperfettanteils oder allgemein eine bewusstere Ernährung. Zusätzlich kann der digitale Kühlschrank dabei helfen, vorhandene Lebensmittel gezielter zu verwenden.

2.2 Large Language Models und Prompt Engineering

Ein Large Language Model, kurz LLM, ist ein Sprachmodell, das auf Grundlage einer Texteingabe eine passende Ausgabe erzeugen kann. Große Sprachmodelle können viele Aufgaben über textuelle Anweisungen bearbeiten, ohne für jede einzelne Aufgabe speziell trainiert zu werden.⁵ In FitFridge wird kein eigenes Modell trainiert und keine Anpassung der Modellgewichte durch Fine-Tuning vorgenommen. Stattdessen wird ein vorhandenes Modell lokal über Ollama genutzt und ausschließlich über den Prompt gesteuert: Ein ausführlicher, iterativ verfeinerter Rezept-Prompt, regelbasierte Backend-Validierung und eine Retry-Schleife übernehmen die Steuerung, die andernfalls über ein Training erfolgen müsste.

Das Standardmodell im finalen Projektstand ist qwen3.5:latest. Zusätzlich sind die kleineren Modelle qwen3:4b und gemma3:1b hinterlegt. Diese dienen ausschließlich schnellen Tests und dem Debugging auf schwächerer Hardware wie Laptops; für die Rezeptgenerierung sind sie zu schwach und liefern häufig unbrauchbare Antworten. Die lokale Ausführung wurde gewählt, weil dadurch keine API-Kosten entstehen und das Projekt unabhängig von einer kostenpflichtigen Cloud-Schnittstelle demonstriert werden kann.

⁵ Vgl. Brown, Tom B. et al. (2020): Language Models are Few-Shot Learners, online verfügbar unter: <https://arxiv.org/abs/2005.14165>, abgerufen am 12.07.2026.

Damit das LLM brauchbare Ergebnisse liefert, wird Prompt Engineering eingesetzt. Der Prompt enthält die konkrete Aufgabe, den Kontext und Regeln für die Antwort. Für FitFridge ist das wichtig, weil das Modell nicht beliebige Rezepte erzeugen soll. Es soll vorhandene Kühlschrankzutaten berücksichtigen, Mengen in Gramm verwenden, Makroziele beachten und strukturierte Daten zurückgeben.

Besonders wichtig ist die strukturierte Ausgabe. Ollama unterstützt strukturierte Ausgaben, bei denen Modellantworten in ein festes Format wie JSON gebracht werden können.⁶ Dadurch kann das Backend die Antwort automatisch weiterverarbeiten. Rezepttitel, Zutaten, Mengen, Kochschritte und Makros können ausgelesen und geprüft werden.

2.3 Kontextgestützte Generierung

Der Rezeptgenerator benötigt aktuelle Kühlschrankdaten. Ohne diesen Kontext könnte das Modell zwar allgemeine Rezepte erzeugen, aber nicht wissen, welche Zutaten tatsächlich vorhanden sind. Deshalb werden die Kühlschrankprodukte des angemeldeten Nutzers aus der Datenbank geladen und in den Prompt eingefügt.

Dieser Ansatz ähnelt Retrieval-Augmented Generation. RAG kombiniert ein generatives Modell mit zusätzlich abgerufenem externem Wissen.⁷ FitFridge nutzt jedoch keine klassische RAG-Architektur mit Vektordatenbank oder semantischer Suche. Passender ist daher die Bezeichnung **RAG-ähnlicher Ansatz** oder **kontextgestützte Generierung**.

Ein vereinfachter Prompt-Kontext kann so aussehen:

1 Hähnchenbrust (165kcal 31P 4F 0C /100g)
2 Reis (130kcal 2P 0F 28C /100g)
3 Paprika (31kcal 1P 0F 6C /100g)

Der Ablauf lässt sich in die drei Schritte Retrieval, Augmentation und Generation einteilen. Zunächst werden die Kühlschrankdaten des angemeldeten Nutzers aus der Datenbank abgerufen. Anschließend werden die verfügbaren Zutaten, IDs und Nährwerte strukturiert in den Prompt eingebettet. Auf dieser Grundlage erzeugt Ollama einen Rezeptvorschlag, der sich ausschließlich an den bereitgestellten Kühlschrankdaten orientieren soll (siehe Abb. 1).

Die Nummern sind dabei laufende IDs, die den Kühlschrankprodukten für diesen Prompt zugewiesen werden. Das LLM soll nicht nur Namen von Zutaten nennen, sondern diese IDs in seiner Antwort referenzieren. Gibt das Modell eine unbekannte ID zurück, erkennt das Backend das Rezept als ungültig. Die verfügbaren Mengen

⁶ Vgl. Ollama: Structured Outputs, online verfügbar unter: <https://docs.ollama.com/capabilities/structured-outputs>, abgerufen am 12.07.2026.

⁷ Vgl. Lewis, Patrick et al. (2020): Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, online verfügbar unter: <https://arxiv.org/abs/2005.11401>, abgerufen am 12.07.2026.

werden bewusst nicht in den Prompt geschrieben; realistische Portionsgrößen und die Einhaltung des Vorrats werden über Prompt-Regeln und die Backend-Validierung sichergestellt.

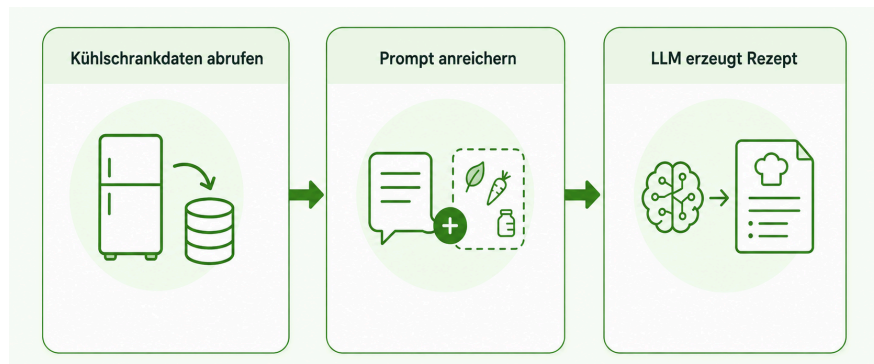


Abbildung 1: RAG-ähnlicher Ansatz
Quelle: Eigene Darstellung

2.4 Adaptive Systeme und MAPE-K

FitFridge kann als kontextadaptives System verstanden werden. Die Ausgabe passt sich an aktuelle Kühlschrankdaten, vorhandene Mengen, Rezeptkategorie und Makroziele an. Die Anwendung lernt dabei nicht langfristig aus Nutzerverhalten, sondern reagiert auf aktuelle Daten.

Ein passendes Konzept aus dem Bereich adaptiver Systeme ist MAPE-K. MAPE-K steht für Monitor, Analyze, Plan, Execute und Knowledge. Das Modell wird in der Literatur als Referenzmodell für Feedback-Schleifen in selbstadaptiven Systemen verwendet.

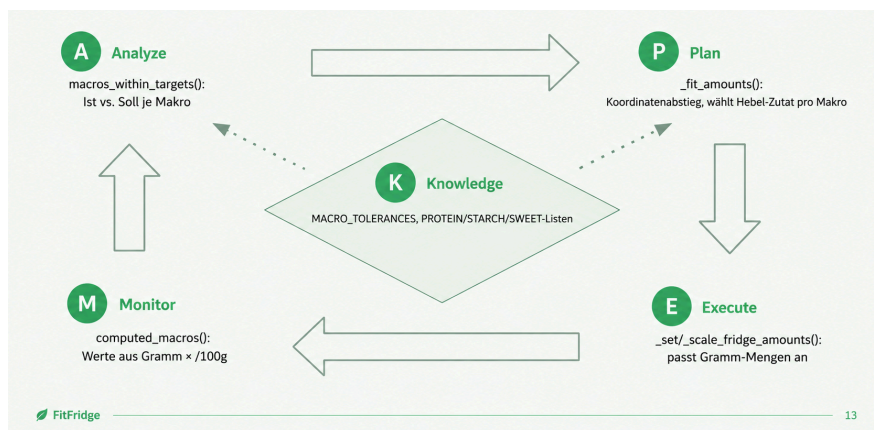


Abbildung 2: MAPE-K: Die Makro-Reparatur
Quelle: Eigene Darstellung

In FitFridge lässt sich MAPE-K besonders gut auf die Makro-Reparatur anwenden. Wenn ein Rezept grundsätzlich sinnvoll ist, die angegebenen Zielwerte jedoch knapp verfehlt, versucht das Backend die Gramm-Mengen einzelner Zutaten anzupassen. Zunächst werden die aktuellen Makronährwerte aus den verwendeten Mengen

berechnet. Anschließend werden diese Ist-Werte mit den Zielwerten verglichen. Auf Basis dieser Abweichung plant das System eine passende Mengenanpassung und führt diese aus. Toleranzbereiche sowie hinterlegte Lebensmittellisten dienen dabei als Wissensbasis (siehe Abb. 2).

3 Projektkontext und technische Grundlage

3.1 Anforderungen an den ASaAI-Teil

Der ASaAI-Teil von FitFridge umfasst zwei Kernfunktionen. Die erste Funktion ist die KI-basierte Nährwertschätzung. Sie ergänzt die Produktsuche und liefert geschätzte Werte für Kalorien, Protein, Fett und Kohlenhydrate pro 100g. Diese Funktion soll den Nutzer entlasten, wenn keine passenden Datenbankwerte verfügbar sind oder eine schnelle Näherung benötigt wird. Die Datenbank enthält überwiegend verarbeitete Markenprodukte. Wer etwa nach „Mango“ sucht, erhält Treffer wie Mango-Lassi oder Mango-Sorbet, statt der Frucht selbst. Die KI-Schätzung betrachtet dagegen ausschließlich den eingegebenen Suchtext und liefert dafür eine Schätzung pro 100 g. Besonders für Obst, Gemüse und Fleisch bietet sich das an, da deren Nährwerte weitgehend konstant sind.

Die zweite Funktion ist der KI-gestützte Freestyle-Rezeptgenerator. Dabei geben Nutzerinnen und Nutzer Zielwerte für Kalorien, Protein, Fett oder Kohlenhydrate an. Es müssen nicht alle vier Werte angegeben werden, mindestens einer ist jedoch erforderlich, beispielsweise genügt ein Protein- und ein Kalorienziel. Zusätzlich wird eine Rezeptkategorie gewählt: Hauptspeise, Frühstück, Abendessen, Nachspeise oder Snack. Anschließend werden auf Basis der gespeicherten Kühlschrankschranklebensmittel Rezeptvorschläge erzeugt. Um lange Ladezeiten zu vermeiden, werden bis zu drei unterschiedliche Vorschläge nacheinander generiert.

Funktional soll ein Rezeptvorschlag Titel, Zutaten, verwendete Kühlschrankschrankprodukte, Mengen, Zubereitungsschritte, berechnete Makros sowie eine kurze Begründung enthalten, warum das Gericht kulinarisch funktioniert. Zusätzlich können erzeugte Rezepte gespeichert werden. Nicht-funktional waren vor allem einfache Bedienung, lokale Ausführung ohne API-Kosten, Nutzertrennung und Datenqualität relevant. Besonders wichtig ist, dass Nährwerte nicht ungeprüft vom Modell übernommen werden. Die finalen Makros werden im Backend berechnet.

3.2 Vorgehensmodell und Aufgabenverteilung

Die Projektorganisation erfolgte nach einem Kanban-orientierten Vorgehen. Aufgaben wurden über Discord abgestimmt und nach Bearbeitungsstand strukturiert. Zusätzlich wurden Git-Feature-Branches genutzt, um einzelne Arbeitspakete getrennt voneinander zu entwickeln. Die Integration in den Main-Branch erfolgte nach Absprache und gegenseitiger Prüfung über einen Merge-Branch. Ergänzend fanden regelmäßige wöchentliche Abstimmungen statt.

Regelmäßige Abstimmungen fanden donnerstags statt. Außerdem gab es eine Zwischenpräsentation und eine Abschlusspräsentation. Nach der Zwischenpräsentation wurde der ASaAI-Teil fokussiert. Ursprünglich gab es sechs KI-Ideen, die jedoch nicht alle stabil funktionierten. Deshalb wurden zwei zentrale Funktionen priorisiert: die KI-basierte Nährwertschätzung und der Freestyle-Rezeptgenerator. Dadurch konnte mehr Zeit in Prompt-Aufbau, LLM-Anbindung, Retry-Mechanismus, Makro-Reparatur und Validierung investiert werden.

Die Projektarbeit wurde von Sriraam Sinnarajah und Vu Thanh Tung Paeper gemeinsam umgesetzt. Beide arbeiteten am Backend und an den ASaAI-Funktionen. Sriraam Sinnarajah arbeitete vor allem an Backend- und ASaAI-bezogenen Funktionen, darunter Rezeptgenerierung, Nährwertschätzung, Validierungslogik und fachliche Ausarbeitung. Vu Thanh Tung Paeper arbeitete ebenfalls am Backend, ASaAI-bezogenen Funktionen und übernahm zusätzlich wesentliche Teile der API-Anbindung, Datenbankstruktur, OpenFoodFacts-Integration, Frontend-Umsetzung und finalen technischen Konsolidierung.

3.3 Tech-Stack und Datenquellen

FitFridge wurde als Webanwendung mit Python 3.10+ und Flask 3 umgesetzt. Für die Datenhaltung wird SQLite verwendet. Das Frontend basiert auf Jinja2-Templates und Vanilla-JavaScript. Für die KI-Funktionen wird Ollama lokal genutzt. Externe Produktdaten kommen über OpenFoodFacts. Ergänzend wurde Claude (Anthropic) als Entwicklungsunterstützung in Bereichen eingesetzt, in denen Vorwissen fehlte, insbesondere bei der Frontend-Umsetzung und Prompt-Tuning.

OpenFoodFacts wird für Produkt- und Nährwertdaten genutzt. Die API stellt unter anderem Informationen zu Zutaten und Nährwerten von Produkten bereit.⁸ Die Rezepte werden ausschließlich vom lokalen LLM generiert.

Die wichtigsten Datenquellen sind:

⁸ Vgl. OpenFoodFacts: API Documentation, online verfügbar unter: <https://openfoodfacts.github.io/openfoodfacts-server/api/>, abgerufen am 12.07.2026.

Tabelle 1: Datenquellen

Quelle: Eigene Darstellung

Datenquelle	Verwendung
Kühlschrankdaten	Grundlage für Rezeptgenerierung
OpenFoodFacts	Produkt- und Nährwertdaten
KI-Schätzung	ergänzende Nährwertquelle
Nutzerziele	Kalorien und Makroziele
Rezeptkategorie	Einschränkung für die Generierung

Die Kühlschrankdaten enthalten unter anderem Produktname, gespeicherte Menge, Einheit und Nährwerte pro 100 g. Die Nährwerte werden dabei nicht als feste Gesamtsumme gespeichert, sondern aus den jeweiligen 100-g-Werten und der aktuell vorhandenen Menge berechnet. Dadurch bleiben die angezeigten Kalorien- und Makrowerte automatisch an die tatsächliche Produktmenge im Kühlschrank angepasst (siehe Abb. 3).



Abbildung 3: Kühlschrank Übersicht, Produktanzeige
Quelle: Eigene Darstellung

4 Entwurf und Implementierung

4.1 ASaAI-Systemarchitektur

Der ASaAI-Teil ist als eigener Pfad innerhalb der Flask-Anwendung umgesetzt. Während der klassische Software-Engineering-Teil vor allem HTML-Formulare, Flask-Routen, Services und Repositories nutzt, arbeitet der ASaAI-Bereich stärker mit **fetch**-Anfragen und JSON. Dadurch werden KI-bezogene Anfragen getrennt verarbeitet und strukturiert an das Frontend zurückgegeben.

Grundlage der gesamten Anwendung ist ein durchgängiges Schichtenmodell: Im Browser laufen Jinja2-Templates und Vanilla-JavaScript, darunter folgen Flask-Routen, Fachlogik-Services und Repositories mit reinem SQL auf einer SQLite-Datenbank. Abhängigkeiten verlaufen ausschließlich von oben nach unten. Diese Architekturentscheidung trennt HTTP-Verarbeitung, Fachlogik und Datenzugriff voneinander und ist der Grund, warum alle 39 automatisierten Tests ohne laufenden Webserver auskommen. Abbildung 4 zeigt dieses Schichtenmodell mit dem Software-Engineering-Teil (links) und dem ASaAI-Teil (rechts).

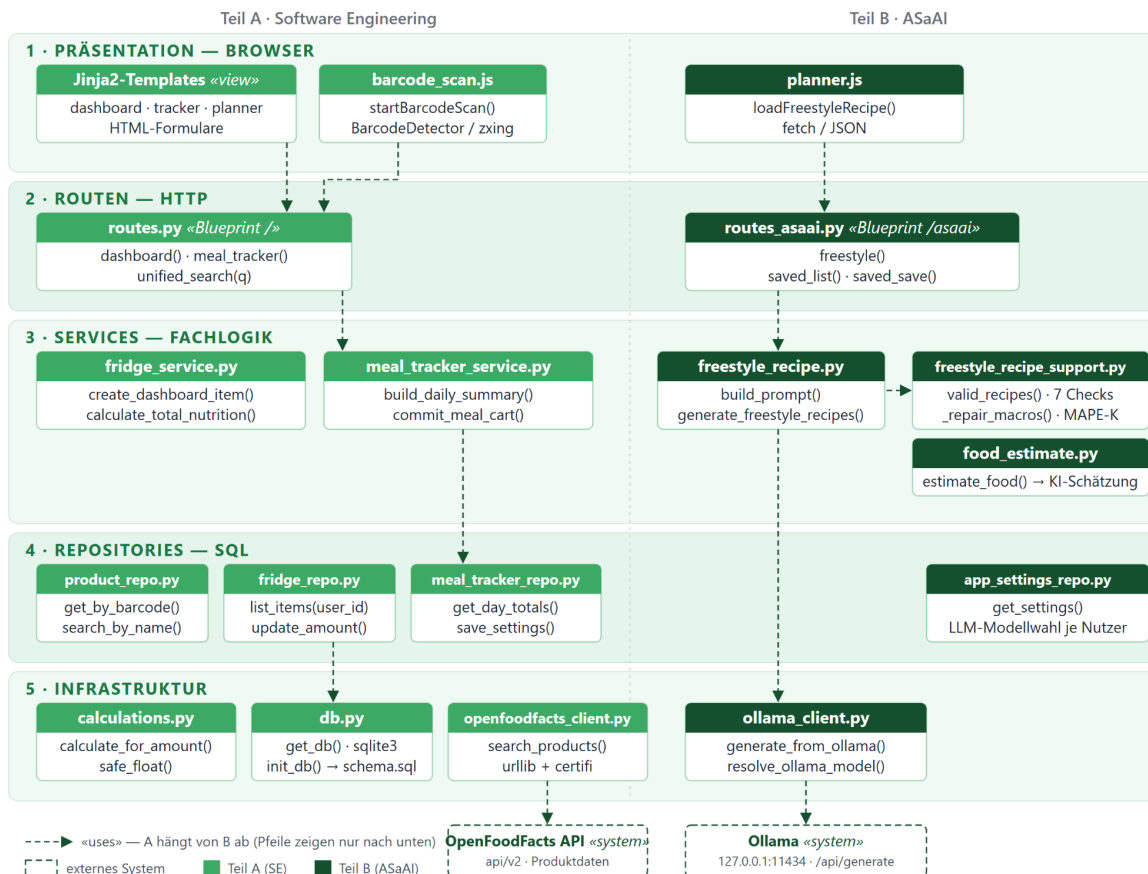


Abbildung 4: Schichtenmodell

Quelle: Eigene Darstellung

Die daraus resultierende statische Struktur der Module dokumentiert das UML-Klassendiagramm in Abbildung 5. Es zeigt die wichtigsten Module mit Attributen, Methoden und «uses»-Abhängigkeiten, darunter auch Querbeziehungen, die im Schichtenmodell nicht sichtbar sind, etwa den Zugriff der Produktsuche (unified_search) auf die KI-Schätzung (food_estimate).

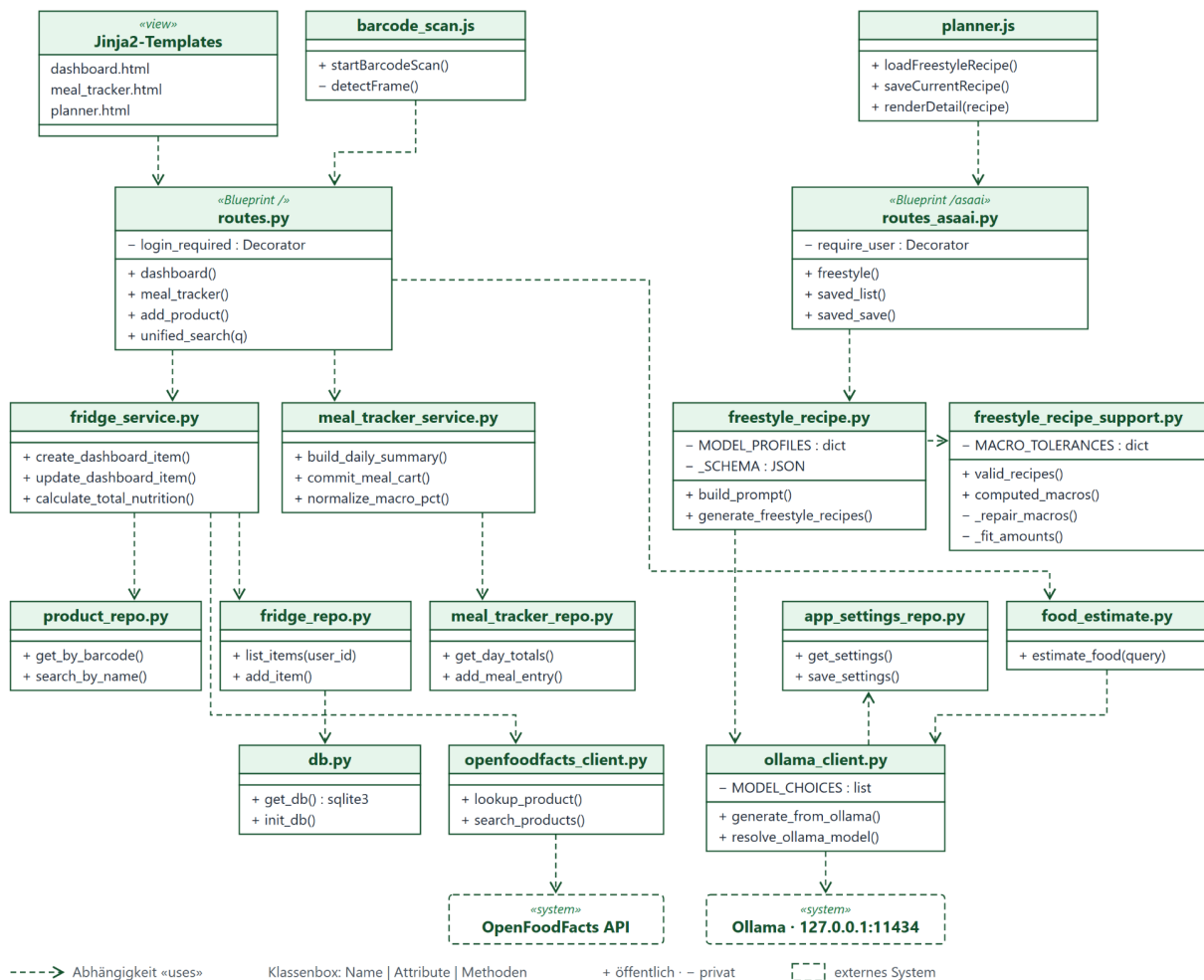


Abbildung 5: UML-Klassendiagramm

Quelle: Eigene Darstellung

4.2 KI-basierte Nährwertschätzung

Die KI-basierte Nährwertschätzung erweitert die Produktsuche von FitFridge. Bei einer Lebensmittelsuche werden nicht nur lokale Datenbankeinträge und OpenFoodFacts-Ergebnisse berücksichtigt, sondern zusätzlich eine KI-generierte Schätzung erzeugt. Diese liefert ungefähre Werte für Kalorien, Protein, Fett und Kohlenhydrate pro 100 g. Dadurch kann das System auch dann einen nutzbaren Vorschlag anzeigen, wenn kein passender Datenbankeintrag vorhanden ist.

Technisch erstellt `estimate_food()` einen Prompt für Ollama. Das Modell soll eine strukturierte Antwort mit Nährwerten erzeugen. Diese Antwort wird anschließend in dasselbe Format gebracht wie andere Produktsuchergebnisse. Dadurch kann die KI-Schätzung in der Oberfläche ähnlich wie ein normaler Treffer angezeigt werden.

Die Funktion ersetzt keine geprüfte Datenquelle. Sie ist eine Ergänzung für Fälle, in denen kein passender Eintrag gefunden wird oder der Nutzer nicht außerhalb der App recherchieren möchte (siehe Abb. 6).

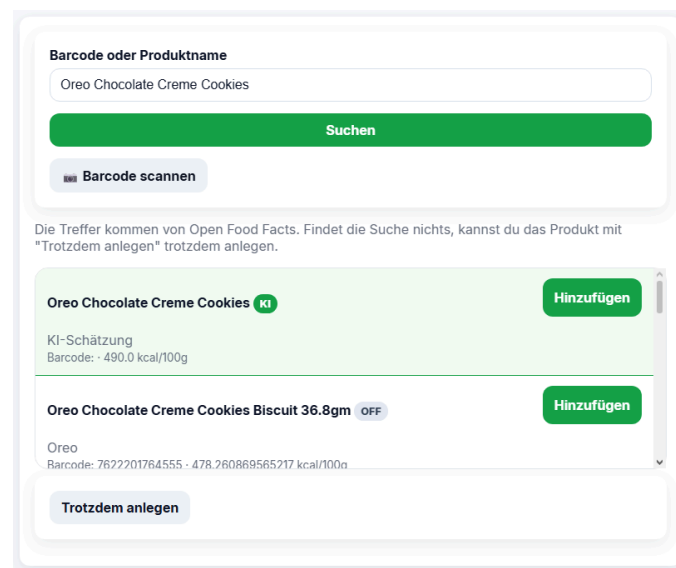


Abbildung 6: KI-Schätzung bei Suche

Quelle: Eigene Darstellung

4.3 KI-gestützter Freestyle-Rezeptgenerator

Der Freestyle-Rezeptgenerator ist die zentrale ASaAI-Funktion von FitFridge. Nutzerinnen und Nutzer geben mindestens ein Makroziel für Kalorien, Protein, Fett oder Kohlenhydrate sowie eine Rezeptkategorie an; es reicht beispielsweise ein Protein- und ein Kalorienziel. Anschließend lädt das System die Kühlschrankprodukte des angemeldeten Nutzers, bereitet diese als strukturierten Kontext auf und integriert sie in den Prompt. Danach wird das lokale LLM über Ollama aufgerufen. Das Rezept wird dabei nicht aus einer bestehenden Rezeptdatenbank geladen, sondern auf Basis der vorhandenen Lebensmittel neu generiert.

Jeder Vorschlag enthält neben Titel, Zutaten, Mengen, Kochschritten und berechneten Makros auch eine kurze Begründung, warum das Gericht kulinarisch zusammenpasst. Zusätzlich können erzeugte Rezepte gespeichert werden. Um längere Wartezeiten zu vermeiden, können bis zu drei unterschiedliche Rezeptvorschläge nacheinander generiert werden.

Abbildung 7 zeigt, wie dieser Vorgang aussieht:

Makroziel → Rezeptkategorie → Generieren → Speichern

Dein Ziel

Protein

Kalorien

60

900

Carbs

Fett

Rezeptart

Hauptspeise

Freestyle-Rezept

Vorschlag

1 Vorschlag · weitere laden...

Hähnchen mit Reis und Brokkoli

LLM

900 kcal · 76g protein · 86g carbs · 26g fat

Hähnchenbrust

Reis

Brokkoli

Olivenöl

Parmesan

Weitere Vc geladen...

Zartes Rumpsteak mit Spaghetti

Rezept speichern

Aus deinem Kühlschrank generiert

950 kcal · 58g protein · 90g carbs · 38g fat

LLM

Steak Spaghetti Olivenöl Parmesan Tomaten Zwiebel Paprika

Warum

Das knusprige, würzige Steak bietet eine sättigende Textur, während die leicht gesalzenen Spaghetti als perfekte neutrale Beilage dienen.

Zutaten

150g Steak
120g Spaghetti
15g Olivenöl
10g Parmesan
100g Tomaten
1g Zwiebel
1g Paprika
Gewürz Salz
Gewürz Pfeffer

Zubereitung

1. Steak waschen, trocken tupfen und kräftig mit Salz und Pfeffer würzen.
2. Spaghetti in Salzwasser nach Packungsanleitung al dente kochen.
3. Während die Nudeln kochen, das Olivenöl in einer Pfanne erhitzen.
4. Steak in der heißen Pfanne bei mittlerer Hitze ca. 3-4 Minuten pro Seite anbraten bis gewünschte Gare.
5. Gemüse (Tomaten, Zwiebel, Paprika) in der letzten Minute der Steak-Zeit kurz mitbraten.
6. Steak aus der Pfanne nehmen und ruhen lassen.
7. Nudeln mit dem angebratenen Gemüse und etwas Bratensaft vermengen.
8. Steak in Scheiben schneiden und mit Nudeln anrichten, Parmesan darüber streuen.

Abbildung 7: Rezeptgenerator
Quelle: Eigene Darstellung

4.4 Prompt-Aufbau und Kontextinjektion

Der Prompt verbindet Nutzereingaben, Kühlschrankdaten und Regeln für die Modellantwort. Er enthält je Kühlschrankprodukt eine laufende ID, den Namen und die Nährwerte pro 100 g. Die vorhandenen Mengen werden nicht in den Prompt geschrieben; realistische Portionen und der Vorratsabgleich werden über Prompt-Regeln und die Backend-Validierung sichergestellt. Der Prompt ist in Regelblöcke gegliedert (Rezeptlogik, Gerichtsart, Zutatenregeln, Mengen, Konsistenzregeln, Qualität, Nährwerte, Ausgabe).

Zusätzlich passt sich der Prompt dynamisch an: Bei gesetzten Zielwerten wird eine zutaten spezifische Makro-Strategie eingefügt (z. B. welche vorhandene Zutat als Protein- oder Stärkehebel dient), bei Wiederholungsversuchen ein Feedback-Hinweis mit dem konkreten Fehlergrund, und bereits vorgeschlagene Titel werden ausgeschlossen.

4.5 Retry-Schleife, Makro-Reparatur und Validierung

LLM-Antworten können fehlerhaft sein. Das Modell kann ungültige IDs verwenden, unpassende Mengen erzeugen, Makroziele verfehlen oder kein sauberes JSON liefern. FitFridge behandelt solche Fälle über eine Pipeline aus Generierung, Reparatur, Validierung und erneutem Versuch (siehe Abb. 8).

Der vereinfachte Ablauf ist:

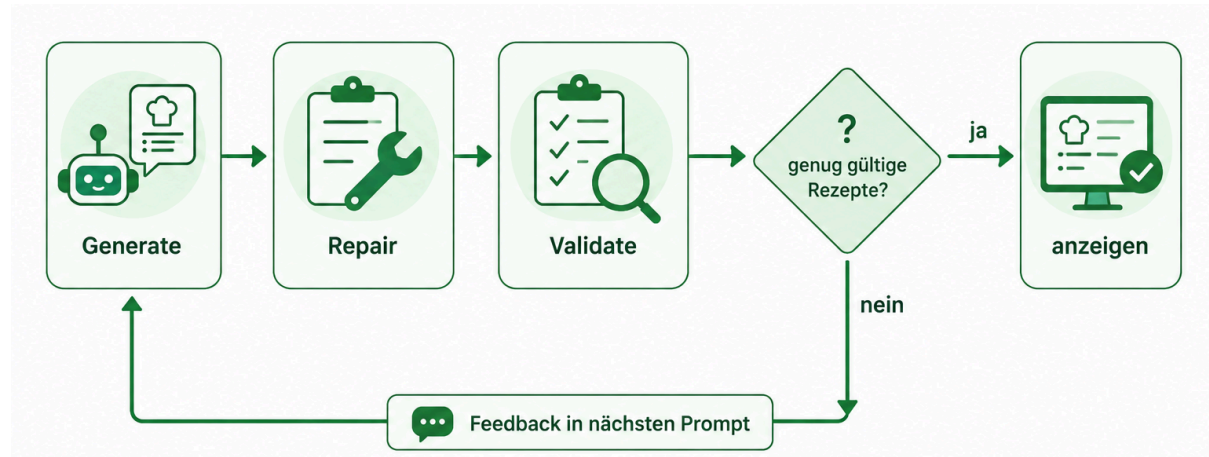


Abbildung 8: Kontrollschicht
Quelle: Eigene Darstellung

Zur Verbesserung der Modellantworten verwendet FitFridge eine Retry-Schleife mit Feedback- Mechanismus. Wenn ein Rezept ungültig ist, wird es nicht direkt angezeigt. Stattdessen analysiert das System die Fehlerursache und kann dem Modell im nächsten Generierungsversuch gezieltes Feedback geben. Zusätzlich werden bereits erzeugte Rezepttitel ausgeschlossen, damit nicht derselbe Vorschlag erneut entsteht. Die Schleife endet, sobald genügend gültige Rezeptvorschläge erzeugt wurden oder die maximale Anzahl an Versuchen erreicht ist.

Ergänzend dazu verfügt FitFridge über eine Makro-Reparatur für nahezu passende Rezepte. Wenn ein Rezept grundsätzlich plausibel ist, die Zielwerte aber knapp verfehlt, versucht das Backend die verwendeten Mengen einzelner Zutaten anzupassen. Enthält ein Rezept beispielsweise zu wenig Protein, kann die Menge einer proteinreichen Zutat erhöht werden. Enthält es zu viele Kalorien, kann eine kalorienreiche Zutat reduziert werden. Nach jeder Anpassung werden die Makrowerte erneut berechnet und geprüft.

Vor der Anzeige eines Rezeptes erfolgt eine regelbasierte Validierung. Dabei wird kontrolliert, ob der Vorschlag fachlich und technisch zulässig ist. Die Validierung umfasst sieben zentrale Prüfungen:

Tabelle 2: Die 7 Prüfungen
Quelle: Eigene Darstellung

Prüfung	Zweck
echte Kühlschranks-IDs	verhindert erfundene Zutaten
reale Portionsgrößen	verhindert absurde Mengen
Pantry-Limits	begrenzt Zusatz-Zutaten
kein Doppel-Protein / Doppel-Stärke	vermeidet unplausible Gerichte
kein Süß-Herzhaft-Mix	erhöht kulinarische Plausibilität
mindestens drei Kochschritte	sichert Mindestqualität
Makros im Zielbereich	prüft Zielerreichung

Zentral ist, dass Modellantworten nie direkt angezeigt werden: Zwischen LLM und Frontend liegt mit `freestyle_recipe_support.py` eine Kontrollschicht, die Antworten prüft, die Nährwerte selbst berechnet und ungültige Ergebnisse verwirft. Den zeitlichen Ablauf zeigen die Sequenzdiagramme: Abbildung 9 für den Software-Engineering-Teil am Beispiel der Produktsuche mit anschließendem Kühlschrank-Hinzufügen, Abbildung 10 für die Freestyle-Rezeptgenerierung mit Retry-Schleife.

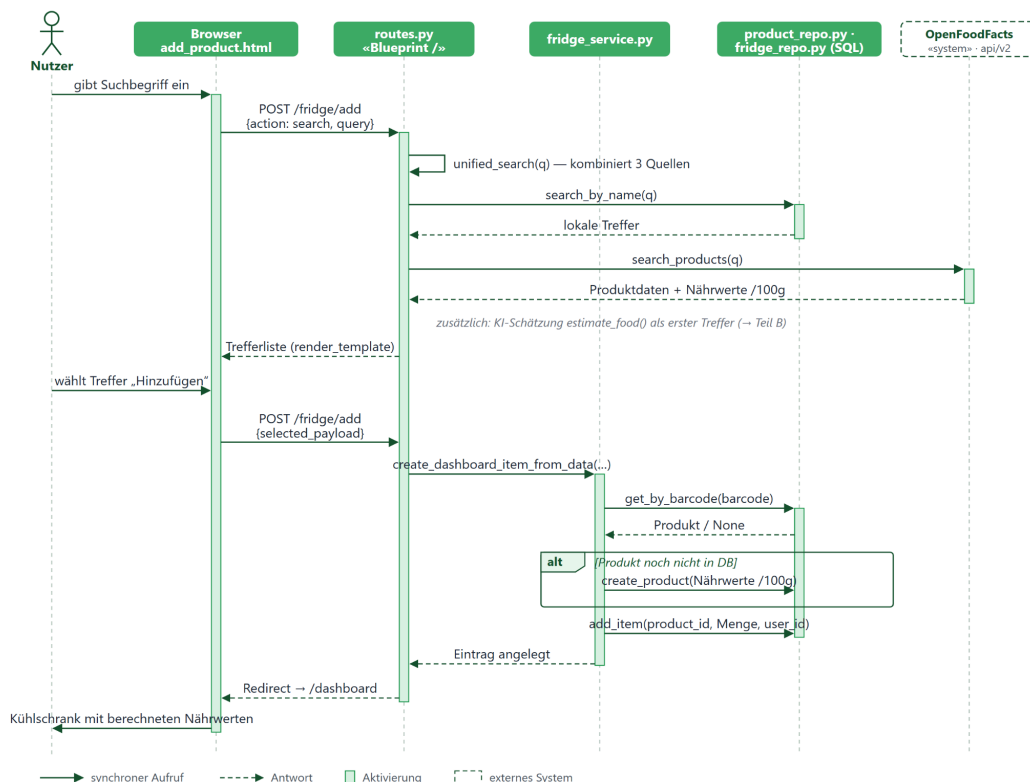


Abbildung 9: Sequenzdiagramm SE
Quelle: Eigene Darstellung

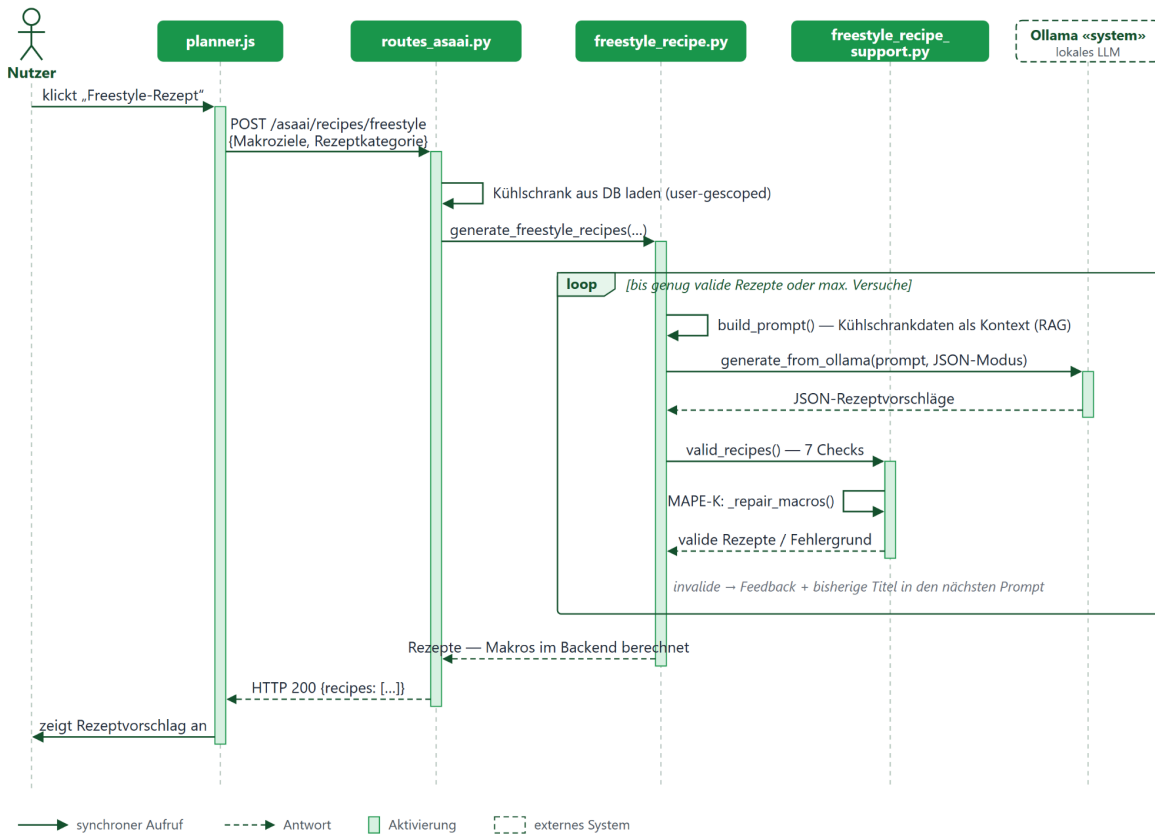


Abbildung 10: Sequenzdiagramm ASaAI

Quelle: Eigene Darstellung

5 Evaluation und Qualitätssicherung

5.1 Teststrategie

Die Qualitätssicherung basiert auf automatisierten Tests und manueller Erprobung. Externe Dienste wie Ollama und OpenFoodFacts werden in den Tests ersetzt. Dadurch laufen die Tests offline und reproduzierbar. Die Testergebnisse hängen damit nicht davon ab, ob ein lokales Modell verfügbar ist oder ob eine externe API antwortet.

Die automatisierten Tests verteilen sich auf mehrere Testdateien und sichern sowohl ASaAI-Funktionen als auch allgemeine Backend-Funktionen ab. Besonders relevant für den ASaAI-Teil sind die Tests zum Freestyle-Rezeptgenerator, zur Ollama-Anbindung und zur Nährwertintegration.

In der Tabelle wird dargestellt, wie die Tests verteilt sind:

Tabelle 3: Testdaten und Anzahl
Quelle: Eigene Darstellung

Testdatei	Anzahl
<code>test_freestyle_recipe.py</code>	14
<code>test_meal_tracker.py</code>	6
<code>test_ollama_client.py</code>	5
<code>test_app_settings.py</code>	5
<code>test_nutrition_integration.py</code>	3
<code>test_product_repo.py</code>	2
<code>test_api_db.py</code>	4

Für den ASaAI-Teil sind vor allem Tests zur Rezeptgenerierung und Ollama-Anbindung relevant. Sie prüfen unter anderem, dass Makros aus Mengen berechnet und nicht vom LLM übernommen werden, dass Low-Carb-Reparaturen funktionieren, dass ungültige Modellantworten abgefangen werden und dass Retry-Mechanismen Teilantworten auffüllen können.

5.2 Bewertung der ASaAI-Funktionen

Die ASaAI-Funktionen können als prototypisch funktionsfähig bewertet werden. Der Rezeptgenerator erzeugt Vorschläge auf Basis von Kühlschranksdaten und Makrozielen. Die KI-Schätzung ergänzt die Produktsuche und reduziert den Aufwand, wenn Datenbankwerte fehlen oder unpassend sind.

Besonders relevant ist der hybride Ansatz. Das LLM wird nicht als alleinige Entscheidungsinstanz verwendet. Es erzeugt Vorschläge, während das Backend kontrolliert, rechnet und validiert. Dadurch wird das Risiko reduziert, fehlerhafte Modellantworten direkt an den Nutzer weiterzugeben.

5.3 Grenzen der Evaluation

Die Evaluation hat klare Grenzen. Es wurde keine umfangreiche Nutzerstudie durchgeführt. Außerdem fand keine ernährungswissenschaftliche Validierung der generierten Rezepte oder KI-Schätzungen statt. Die Schätzung von Nährwerten bleibt daher eine Annäherung.

Die Qualität der Ergebnisse hängt zusätzlich vom verwendeten Modell und von der lokalen Hardware ab. Größere Modelle können bessere Antworten liefern, benötigen

aber mehr Rechenleistung. Kleinere Modelle sind leichter auszuführen, dienen im Projekt aber nur für einfache Debug-Läufe und die KI-Schätzung; für die Rezeptgenerierung sind sie ungeeignet.

Das Demo-Setup ist ebenfalls begrenzt. Im aktuellen Projektstand wird die Datenbank bei jedem Serverstart neu aufgebaut und mit Demo-Daten gefüllt. Für den Prototyp ist dieses Vorgehen ausreichend, da dadurch ein reproduzierbarer Testzustand entsteht. Für ein reales Produkt wäre diese Lösung jedoch nicht geeignet. Dort müsste eine persistente Datenbank eingesetzt werden, bei der Nutzerdaten dauerhaft gespeichert bleiben und Änderungen an der Datenbankstruktur über Migrationen kontrolliert durchgeführt werden.

6 Diskussion

6.1 Bewertung des hybriden Ansatzes

Die zentrale Stärke von FitFridge liegt in der Kombination aus generativer KI und klassischer Backend-Logik. Das LLM eignet sich für kreative Aufgaben wie Rezeptideen, Titel und Kochschritte. Für präzise Aufgaben wie Nährwertberechnung, Nutzertrennung und Mengenprüfung ist deterministische Programmlogik besser geeignet.

Gerade im Ernährungskontext ist diese Trennung wichtig. Ein Rezeptvorschlag ist nur dann nützlich, wenn Zutaten und Makros plausibel sind. Würde die Anwendung alle Angaben ungeprüft vom Modell übernehmen, könnten falsche Nährwerte oder nicht vorhandene Zutaten angezeigt werden. Die Backend-Validierung reduziert dieses Risiko.

6.2 Risiken und Verbesserungspotenziale

Trotz Validierung bleiben Risiken bestehen. Das LLM kann weiterhin unpassende Vorschläge erzeugen oder bei schwierigen Eingaben keine brauchbare Antwort liefern. Die KI-basierte Nährwertschätzung kann ungenau sein und sollte nicht als geprüfte Produktangabe verstanden werden. Auch OpenFoodFacts-Daten können unvollständig oder uneinheitlich sein.

Ein weiteres Risiko ist die Abhängigkeit von lokaler Hardware. Ollama muss laufen und ein passendes Modell muss installiert sein. Auf schwächeren Geräten können Antwortzeiten lang sein; die kleinen Modelle eignen sich dort nur für die KI-Schätzung oder Debug-Zwecke, nicht für ganze Rezeptgenerierungen. Außerdem ist FitFridge kein medizinisches System und keine professionelle Ernährungsberatung. Andere Komplikationen können auch bei unlogischen Makrozielkombinationen, etwa 100 g Protein bei nur 300 kcal, entstehen. Es kann kein Rezept erstellt werden und das System zeigt in diesem Fall den Hinweis

„Makro-Kombination nicht erreichbar“ mit einer Anpassungsempfehlung an, statt ein geschöntes Rezept auszugeben. Die Anwendung ist daher als Prototyp zur Unterstützung fitnessorientierter Mahlzeitenplanung zu verstehen.

Zukünftige Versionen könnten Nutzerpräferenzen stärker berücksichtigen, beispielsweise vegetarisch, vegan, halal, Allergien, Unverträglichkeiten, Kochzeit, Budget oder Lieblingszutaten. Auch Nutzerfeedback wäre sinnvoll. Wenn Nutzer Rezepte speichern, löschen oder bewerten, könnte das System langfristig bessere Vorschläge erzeugen. Technisch wären eine persistente Datenbank, Prompt-Versionierung, besseres Logging und eine systematischere Evaluation sinnvoll.

7 Fazit

7.1 Beantwortung der Forschungsfrage

FitFridge zeigt, wie ein lokal ausgeführtes Large Language Model in eine Ernährungs-Webanwendung integriert werden kann. Die Anwendung verbindet einen digitalen Kühlschrank, Makroziele, KI-basierte Nährwertschätzung und einen KI-gestützten Freestyle-Rezeptgenerator.

Wie können ein lokal ausgeführtes Large Language Model und regelbasierte Backend-Validierung kombiniert werden, um in einer Ernährungs-Webanwendung plausible Rezeptvorschläge und Nährwertschätzungen auf Basis vorhandener Lebensmittel zu erzeugen?

Der Prototyp zeigt, dass diese Kombination grundsätzlich möglich ist. Das LLM erzeugt Rezeptideen und Nährwertschätzungen. Das Backend prüft Zutaten, Mengen und Makros, berechnet die finalen Nährwerte selbst und verwirft ungültige Antworten. Dadurch entsteht ein kontrollierter hybrider Ansatz.

Die wichtigste Erkenntnis ist, dass generative KI im Ernährungskontext hilfreich sein kann, aber nicht ohne Validierung eingesetzt werden sollte. Für kreative Vorschläge ist das LLM geeignet. Für Präzision, Berechnung und Sicherheit bleibt klassische Backend-Logik notwendig.

7.2 Lessons Learned

Im Verlauf des Projekts wurde deutlich, dass der Einsatz eines Large Language Models allein nicht ausreicht, um zuverlässige Ergebnisse in einer Ernährungsanwendung zu erzeugen. Das LLM kann kreative Rezeptideen liefern und flexibel auf unterschiedliche Eingaben reagieren. Gleichzeitig zeigte sich aber, dass Modellantworten fehlerhaft, unvollständig oder fachlich ungenau sein können.

Besonders bei Nährwerten, Mengenangaben und Zutaten darf die Ausgabe daher nicht ungeprüft übernommen werden.

Eine zentrale Erkenntnis war deshalb, dass ein hybrider Ansatz notwendig ist. Das LLM eignet sich für die Generierung von Rezeptvorschlägen, während klassische Backend-Logik für Kontrolle, Berechnung und Validierung verantwortlich sein muss. Die Makronährwerte werden deshalb nicht aus der Modellantwort übernommen, sondern aus den gespeicherten Produktdaten und den verwendeten Mengen berechnet. Dadurch wurde klar, dass KI im Projekt eher als unterstützende Komponente und nicht als alleinige Entscheidungsinstanz eingesetzt werden sollte.

Außerdem wurde gelernt, dass eine klare Begrenzung des Funktionsumfangs wichtig ist. Zu Beginn gab es mehrere KI-Ideen, die jedoch nicht alle stabil genug umgesetzt werden konnten. Durch die spätere Fokussierung auf den Freestyle-Rezeptgenerator und die KI-basierte Nährwertschätzung konnte mehr Zeit in die technische Qualität, die Validierung und die Verständlichkeit der Funktionen investiert werden. Für ein Hochschulprojekt war diese Reduktion sinnvoller als eine größere Anzahl unausgereifter KI-Funktionen.

Ein weiterer Lernpunkt war die Bedeutung von Prompt Engineering. Schon kleine Änderungen im Prompt können beeinflussen, ob das Modell gültiges JSON, passende Zutaten oder realistische Mengen erzeugt. Gleichzeitig wurde deutlich, dass Prompt Engineering allein nicht zuverlässig genug ist. Deshalb mussten Retry-Mechanismen, Feedback und regelbasierte Prüfungen ergänzt werden.

Auch die lokale Nutzung von Ollama brachte wichtige Erfahrungen. Der Vorteil lag darin, dass keine kostenpflichtige Cloud-API benötigt wurde und verschiedene Modelle lokal getestet werden konnten. Gleichzeitig wurde sichtbar, dass lokale Modelle je nach Hardware langsamer reagieren und qualitativ schwanken können.

Organisatorisch zeigte das Projekt, dass regelmäßige Abstimmungen und eine klare Aufgabenstruktur wichtig sind. Durch Kanban-orientiertes Arbeiten, Git-Branches und wöchentliche Abstimmungen konnten Aufgaben verteilt und Zwischenergebnisse zusammengeführt werden. Gleichzeitig wurde deutlich, dass Überschneidungen bei der Entwicklung entstehen können, wenn Aufgaben nicht früh genug eindeutig abgegrenzt werden. Für zukünftige Projekte wäre eine noch klarere technische Planung zu Beginn hilfreich.

Insgesamt wurde gelernt, dass generative KI in einer Webanwendung besonders dann sinnvoll eingesetzt werden kann, wenn sie durch strukturierte Daten, klare Prompts und regelbasierte Backend-Kontrollen ergänzt wird. FitFridge zeigt damit, dass der praktische Nutzen nicht allein durch das LLM entsteht, sondern vor allem durch die Kombination aus KI, Datenbasis, Validierung und klassischer Softwareentwicklung.

Literaturverzeichnis / Fußnoten

- [1] Yazio: Calorie Counter App. URL: <https://www.yazio.com/en/calorie-counter>
Yazio nennt unter anderem Kalorienzählen, Nährstofftracking und Barcode-Scanner als Funktionen.
- [2] MyFitnessPal: Calorie Tracker & Macro Tracking. URL:
<https://www.myfitnesspal.com/>
MyFitnessPal beschreibt sich als App für Ernährungs-, Kalorien- und Makrotracking.
- [3] Brown, T. B. et al. (2020): *Language Models are Few-Shot Learners*. URL:
<https://arxiv.org/abs/2005.14165>
- [4] Ollama: Structured Outputs. URL:
<https://docs.ollama.com/capabilities/structured-outputs>
Ollama beschreibt strukturierte Ausgaben zur Erzwingung eines JSON-Schemas für Modellantworten.
- [5] Lewis, P. et al. (2020): *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. URL: <https://arxiv.org/abs/2005.11401>
- [6] Arcaini, P.; Riccobene, E.; Scandurra, P. (2015): *Modeling and Analyzing MAPE-K Feedback Loops for Self-Adaptation*. URL:
https://cs.unibg.it/scandurra/papers/seams2015_cameraReady.pdf
- [7] OpenFoodFacts: API Documentation. URL:
<https://openfoodfacts.github.io/openfoodfacts-server/api/>
Die API ermöglicht das Abrufen von Produktinformationen wie Zutaten und Nährwerten.

Abbildungsverzeichnis

Abb. 1: RAG-ähnlicher Ansatz.....	7
Abb. 2: MAPE-K: Die Makro-Reparatur.....	7
Abb. 3: Kühlschrank Übersicht, Produktanzeige.....	10
Abb. 4: Schichtenmodell.....	11
Abb. 5: UML-Klassendiagramm.....	12
Abb. 6: KI-Schätzung bei Suche.....	13
Abb. 7: Rezeptgenerator.....	14
Abb. 8: Kontrollschicht.....	15
Abb. 9: Sequenzdiagramm SE.....	16
Abb. 10: Sequenzdiagramm ASaAI.....	17